

Web Attack Forensics

Apache access logs · ModSecurity audits · breach timelines

Stefanus Eko Prasetyo

Lab-4 · hands-on session

```
root@dfir:/var/log/apache2#  
# cat error.log | grep SQLi  
ModSecurity: Warning. detected SQLi  
using libinjection  
[id "942100"] ... CRITICAL
```

Today's roadmap

1

The web server as witness

what Apache records about every request it serves

2

The lab environment

a deliberately vulnerable Docker app - and why we isolate it

3

Three log files

access.log, error.log, and the ModSecurity audit trail

4

Three attack classes

path traversal, remote command execution, SQL injection

5

Decoding payloads

URL encoding, double encoding, and base64 in queries

6

Challenges & working paper

upstream breach case + two new challenges + documentation rules

Every request leaves three records

Log file	Written by	Records
access.log	Apache	one line per request: IP, time, method, URI, status, size, user agent, referer
error.log	Apache + PHP + ModSecurity	application errors, permission failures, WAF warnings
modsec_audit.log	ModSecurity (WAF)	full transactions: request AND response bodies, matched rules

Found under /var/log/apache2/ on the server (or inside the Docker container: `docker exec -it <id> bash`).

A vulnerable app - on purpose, in Docker

```
root@dfir:/var/log/apache2#  
$ git clone https://github.com/stprasst/digital-forensic-lab  
$ cd digital-forensic-lab/Lab\ 4/files/app  
$ docker build -t app:latest .  
$ docker run -p 9090:80 app:latest  
  
# browse http://127.0.0.1:9090/  
$ docker ps -q          # container id  
18c7468edbe7  
$ docker exec -it 18c7468edbe7 bash  
root@18c7468edbe7:/# cd /var/log/apache2
```

The four pages

- ▶ images.php + view.php - image viewer (path traversal)
- ▶ command.php - runs what you type (RCE)
- ▶ users.php - user search (SQL injection)
- ▶ NEVER deploy or expose this app - it is a crime scene, not a product

access.log — one line, nine fields

root@dfir:/var/log/apache2#

```
192.168.0.106 - - [16/Feb/2023:01:35:27 +0500] "GET /view.php?image=../../../../../etc/passwd
HTTP/1.1" 200 650 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:102.0)"
# IP      - user [time +zone] "METHOD URI PROTO" status size "referer" "agent"
```

Field	Forensic value
client IP + timestamp	who, and exactly when (mind the timezone)
method + URI	what they asked for - payloads usually live right here
status + size	200 with a big size = the attack WORKED; 404s = scanning
user agent	browser fingerprint - or a tool announcing itself (sqlmap, dirb)

error.log — where failures confess

```
root@dfir:/var/log/apache2#  
# cat error.log | grep "Permission denied"  
PHP Warning: file_get_contents(images/../../../../..  
/etc/shadow): Failed to open stream: Permission denied  
in /var/www/html/view.php on line 10  
  
# cat error.log | grep SQLi  
ModSecurity: Warning. detected SQLi using  
libinjection with fingerprint 's&1c'  
[id "942100"] [severity "CRITICAL"]
```

What error.log gives you

- ▶ PHP warnings expose the failing code path (file + line)
- ▶ Permission denied = the attack was TRIED and the OS stopped it
- ▶ ModSecurity duplicates its warnings here - grep-able
- ▶ great for triage; the audit log has the full detail

modsec_audit.log — the full transcript

```
root@dfir:/var/log/apache2#  
  
--412fc70c-A-- # header: time, IPs, ports, unique id  
--412fc70c-B-- # request headers (browser, cookies)  
--412fc70c-C-- # request BODY (POST data - commands!)  
--412fc70c-F-- # response headers (final status)  
--412fc70c-E-- # response BODY (what the server sent back)  
--412fc70c-H-- # audit trailer: matched rules, scores  
--412fc70c-Z-- # end of transaction  
  
# C and E are the gold: inputs AND outputs, verbatim
```

Reading a transaction

- ▶ sections share one transaction id (412fc70c here)
- ▶ B + C = what the attacker sent
- ▶ E = what the server answered - did it work?
- ▶ H = which CRS rules fired and the anomaly score

OWASP CRS — the rule IDs that matter

Rule ID	Family	Meaning
920350	protocol	Host header is a numeric IP address (WARNING only)
930100 / 930110	LFI	Path traversal pattern in the raw / decoded URI
930120	LFI	OS file access attempt (etc/passwd, etc/shadow)
932150 / 932160	RCE	Unix command injection / shell code found
942100	SQLi	SQL injection detected via libinjection fingerprint
980130	correlation	Inbound anomaly score exceeded - the attack summary

The 980130 line adds up the whole transaction: 'Total Inbound Score: 43 - SQLI=0, XSS=0, RFI=0, LFI=35, RCE=5' - a ready-made verdict.

Path traversal — climbing out of the root

```
root@dfir:/var/log/apache2#  
  
# the ladder an attacker walks up:  
GET /view.php?image=../etc/passwd          200 (202)  
GET /view.php?image=../../etc/passwd       200 (202)  
GET /view.php?image=../../..etc/passwd     200 (203)  
GET /view.php?image=../../../../etc/passwd 200 (650)  
  
# encoded variant in access.log:  
GET /images.php?file=..%2F..%2F..%2Fetc%2Fpasswd  
# WAF: [id 930100] Path Traversal Attack (/../)
```

Signatures

- ▶ repeated URIs differing only in ../ depth
- ▶ size jump = the file actually opened
- ▶ %2F is just / URL-encoded - always decode
- ▶ reading is the impact; LFI would also execute

Remote command execution — the shell behind the form

```
root@dfir:/var/log/apache2#  
  
# access.log hides the payload (POST):  
POST /command.php HTTP/1.1" 200 1143  
  
# modsec_audit.log C-section shows it:  
cmd=whoami  
cmd=ls+-la+/          # spots database.sql!  
cmd=cat+%2Fdatabase.sql # full user table  
cmd=cat+%2Fimportant_note.txt  
  
# E-section returns the output; server runs as www-data
```

Signatures

- ▶ POSTs to one endpoint with growing sizes
- ▶ C-section: cmd=, ;, |, backticks, base64 blobs
- ▶ E-section mirrors command output (file listings)
- ▶ error.log: 'Permission denied' = OS held the line

SQL injection — asking the database questions

```
root@dfir:/var/log/apache2#  
  
# boolean probe pair (does it filter?):  
user2' and 1=1 #      -> normal results  
user2' and 1=2 #      -> results vanish  
  
# the dump:  
' union select username, password from users #  
  
# automated: the user agent confesses  
"sqlmap/1.6.11#stable (https://sqlmap.org)"  
# WAF: 942100 libinjection fingerprint 's&1c'
```

Signatures

- ▶ quotes + SQL keywords in parameters
- ▶ true/false probe pairs = blind SQLi testing
- ▶ UNION SELECT = data extraction
- ▶ tool UAs (sqlmap) and random probe args (HOvh)

Decoding the payload — sometimes twice

```
root@dfir:/var/log/apache2#  
  
# one layer:  
..%2F..%2Fetc%2Fpasswd -> ../../etc/passwd  
%27+OR+1%3D1-- -> ' OR 1=1--  
  
# two layers (evade naive filters):  
ctf%257Bd0ub13%257D  
  1st decode -> ctf%7Bd0ub13%7D  
  2nd decode -> ctf{d0ub13}  
  
$ python3 -c "import urllib.parse as u;  
  print(u.unquote_plus(u.unquote_plus(S)))"
```

Decode rules

- ▶ %XX = one byte; + or %20 = space
- ▶ decode until it stops changing
- ▶ base64 may hide underneath (Q1 of the case!)
- ▶ CyberChef: URL decode twice, then Magic

Rebuilding the breach timeline

- 1 Rank IPs and endpoints (awk + sort + uniq -c)
- 2 Flag anomalies: 404 bursts, size jumps, odd UAs
- 3 Open the audit log at the interesting timestamps
- 4 Decode every payload fully (URL, base64, char())
- 5 Correlate the three logs into one story

```
root@dfir:/var/log/apache2#  
  
# who is noisy?  
$ awk '{print $1}' access.log | sort | uniq -c  
  | sort -rn | head  
38 192.168.0.106  
  
# status codes:  
$ awk '{print $9}' access.log | sort | uniq -c  
  
# biggest responses:  
$ awk '{print $10}' access.log | sort -rn | head
```

The log-triage toolbox

Tool / pattern	Purpose
<code>awk '{print \$1,\$9}'</code>	project IP and status columns from access.log
<code>sort uniq -c sort -rn</code>	frequency ranking - IPs, URIs, status codes, agents
<code>grep -E 'pattern1 pattern2'</code>	multi-signature search across all three logs
<code>docker exec -it <id> bash</code>	shell inside the container to reach /var/log/apache2
<code>python3 + urllib.parse</code>	URL decoding (unquote_plus), base64, char() sequences
CyberChef	interactive decode chains when you don't want to script

Your challenges

Challenge A

the breach case

`files/logs.zip`

- ▶ 38 requests, one attacker
- ▶ 3 attack classes, 10 questions
- ▶ a secret note and a taunt
- ▶ solved together in the Deep Dive

from the original lab

Challenge B — new

`traffic_shop.log`

`files/traffic_shop.log`

- ▶ 1,488 lines of shop traffic
- ▶ a scanner, a probe pair, a dump
- ▶ a goodbye query hiding a flag
- ▶ decode it - more than once

access.log triage

Challenge C — new

`waf_alert.log`

`files/waf_alert.log`

- ▶ four audit transactions
- ▶ sections A to Z, one carries gold
- ▶ the WAF watched everything
- ▶ why did nothing get blocked?

audit log deep read

Flag format: `ctf{xxxx}` (Challenge A keeps the upstream flag format)

Working paper & evidence rules

1 Pre-lab first

section 1.4 of the working paper, before the session

2 Work on copies

unzip logs.zip to your working directory - never edit in place

3 Keep the container isolated

the lab app is deliberately vulnerable: localhost only, kill it after

4 Number your evidence

screenshots and outputs get B-01, B-02, C-01 in the evidence log

5 Cite line-level evidence

answers name the log file and quote the exact line or rule ID

```
root@dfir:/var/log/apache2#  
# evidence log entry  
[B-06] 2026-10-14 08:05  
      grep dirb traffic_shop.log | wc -l  
      65  # scanner burst found  
      -> attacker used dirb/2.22
```

No line, no answer.

every claim must trace to a quoted log line

KEY TAKEAWAYS

01

Three logs, one event

access.log shows the request, error.log the failure, modsec_audit the full transcript

02

POST bodies live in the WAF

access.log hides form payloads - section C is where commands surface

03

Sizes and codes tell outcomes

a 200 with a huge byte count means the dump worked

04

Decode until it stops changing

URL once, URL twice, base64, char() - payloads hide in layers

Read the logs. Rebuild the story.



root@dfir:/var/log/apache2#

```
$ git clone https://github.com/stprasst/digital-forensic-lab
$ cd digital-forensic-lab/Lab\ 04/files
$ ls
logs.zip  traffic_shop.log  waf_alert.log
app/  images/
```